

Automata Simulator Tool

Theory of Computation (TOC)

Academic Project Report

Prepared by: Yash Lund

Technology: Python + Streamlit

Problems Covered: 12 (DFA & NFA)

Pure Python Implementation - No External Automata Libraries

Table of Contents

1. Introduction
2. Objectives
3. Technology Stack
4. System Architecture
5. Automata Problems (12 Problems)
 - 5.1 - 5.12 Individual Problem Details
6. NFA to DFA Conversion
7. Test Results
8. Features Summary
9. Conclusion

1. Introduction

This project implements a comprehensive Automata Simulator Tool for the Theory of Computation (TOC) course. The tool supports simulation of both Deterministic Finite Automata (DFA) and Nondeterministic Finite Automata (NFA) across 12 different language recognition problems.

The simulator provides step-by-step visualization of state transitions, formal 5-tuple definitions, transition tables, state diagrams (SVG), NFA-to-DFA conversion via subset construction, JFLAP file export, and automated test case validation.

The project is built entirely in pure Python with no external automata libraries. The frontend is powered by Streamlit, providing an interactive web-based interface.

2. Objectives

1. Design and implement DFA and NFA for 12 language recognition problems.
2. Provide step-by-step string simulation with transition traces.
3. Display formal definitions: $M = (Q, \Sigma, \delta, q_0, F)$.
4. Generate visual state transition diagrams (SVG).
5. Implement NFA to DFA conversion using subset construction.
6. Export JFLAP-compatible .jff files for each automaton.
7. Provide comprehensive test cases for validation.
8. Build an interactive web UI with Streamlit.

3. Technology Stack

Language: Python 3.x

Frontend: Streamlit (web UI framework)

Backend: Pure Python (automata.py - single file, no external libraries)

Diagram: SVG generation (inline, no dependencies)

Export: JFLAP XML (.jff format)

Report: fpdf2 (PDF generation)

Version Control: Git + GitHub

4. System Architecture

The project follows a clean two-file architecture:

automata.py (Backend)

- Automaton definitions (12 problems)
- DFA simulator (step-by-step)
- NFA simulator (subset tracking)
- Formal definition printer
- Transition table generator
- SVG state diagram generator
- NFA-to-DFA converter (subset construction)
- JFLAP XML generator
- State explanations (viva helper)

- Test case runner
- Interactive CLI menu

app.py (Frontend)

- Streamlit web application
- 7 interactive tabs
- Custom dark theme CSS
- Real-time simulation
- SVG diagram rendering
- JFLAP .jff file download

5. Automata Problems

The simulator covers 12 problems across DFA and NFA types. Each problem below includes its formal definition, transition table, state explanations, and test cases.

5.1 Strings ending with '01' (DFA)

```
M = (Q, Sigma, delta, q0, F)
Q   = { q0, q1, q2 }
Sigma = { 0, 1 }
q0   = q0
F     = { q2 }
```

```
delta(q0, 0) = q1
delta(q0, 1) = q0
delta(q1, 0) = q1
delta(q1, 1) = q2
delta(q2, 0) = q1
delta(q2, 1) = q0
```

DFA for strings over {0,1} that END with '01'.

The automaton remembers only the last 1?2 relevant symbols.

q0 [START]

INITIAL state.

Meaning : No useful suffix seen yet.

Reached : At the start, or after reading '1' from q0 or q2.

Accept? : No

q1

Meaning : The LAST symbol read was '0'.

Reached : After reading '0' from q0, q1, or q2.

Accept? : No (need '1' next to complete '01')

q2 [ACCEPT]

ACCEPT state.

Meaning : The last TWO symbols were '0' then '1'.

The string currently ends with '01'.

Reached : After reading '1' from q1.

Accept? : YES ?

Accepted: '01', '001', '101', '1101', '0101'

Rejected: '0', '10', '00', '11', '110'

5.2 Strings containing 'aba' (NFA)

```
M = (Q, Sigma, delta, q0, F)
Q   = { q0, q1, q2, q3 }
Sigma = { a, b }
q0   = q0
F     = { q3 }
```

```
delta(q0, a) = { q0, q1 }
delta(q0, b) = { q0 }
delta(q1, a) = { q1 }
delta(q1, b) = { q2 }
delta(q2, a) = { q3 }
delta(q2, b) = { q0 }
delta(q3, a) = { q3 }
delta(q3, b) = { q3 }
```

NFA for strings over {a,b} that CONTAIN 'aba'.

Nondeterminism lets the automaton guess where 'aba' starts.

q0 [START]

INITIAL state.

Meaning : 'aba' not found yet; no active match in progress.

Accept? : No

q1

Meaning : Matched the FIRST 'a' of the target 'aba'.

Reached : From q0 on reading 'a' (one of two nondeterministic branches).

Accept? : No

q2

Meaning : Matched 'ab' ? first two characters of 'aba'.

Reached : From q1 on reading 'b'.

Accept? : No

q3 [ACCEPT]

ACCEPT state (trap / sink).

Meaning : 'aba' has been found as a substring.

Once here, all future input keeps it here.

Reached : From q2 on reading 'a'.

Accept? : YES ?

Accepted: 'aba', 'aaba', 'abab', 'bbaba', 'babab'

Rejected: '', 'ab', 'ba', 'bb', 'aab'

5.3 Binary numbers divisible by 3 (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{ q_0, q_1, q_2 \}$

$\Sigma = \{ 0, 1 \}$

$q_0 = q_0$

$F = \{ q_0 \}$

$\delta(q_0, 0) = q_0$

$\delta(q_0, 1) = q_1$

$\delta(q_1, 0) = q_2$

$\delta(q_1, 1) = q_0$

$\delta(q_2, 0) = q_1$

$\delta(q_2, 1) = q_2$

DFA for binary strings whose integer value is divisible by 3.

States track remainder = (current_value) mod 3.

Reading bit b transforms remainder r into $(2r + b) \bmod 3$.

q0 [START] [ACCEPT]

INITIAL & ACCEPT state.

Meaning : Current binary value = 0 (mod 3) ? divisible by 3.

Accept? : YES ? (empty string = value 0, which is divisible by 3)

q1

Meaning : Current binary value = 1 (mod 3).

Accept? : No

q2

Meaning : Current binary value = 2 (mod 3).

Accept? : No

Accepted: '0', '11', '110', '1001', '1100'

Rejected: '1', '10', '100', '111', '1010'

5.4 Even number of a's and b's (DFA)

```

M = (Q, Sigma, delta, q0, F)
Q   = { q0, q1, q2, q3 }
Sigma = { a, b }
q0   = q0
F    = { q0 }

```

```

delta(q0, a) = q1
delta(q0, b) = q2
delta(q1, a) = q0
delta(q1, b) = q3
delta(q2, a) = q3
delta(q2, b) = q0
delta(q3, a) = q2
delta(q3, b) = q1

```

DFA for strings over {a,b} with EVEN count of a's AND EVEN count of b's.

Each state encodes a parity pair (parity-of-a, parity-of-b).

Reading 'a' flips a-parity; reading 'b' flips b-parity.

q0 [START] [ACCEPT]

INITIAL & ACCEPT state.

Meaning : count(a) is EVEN and count(b) is EVEN.

Accept? : YES ?

q1

Meaning : count(a) is ODD and count(b) is EVEN.

Accept? : No

q2

Meaning : count(a) is EVEN and count(b) is ODD.

Accept? : No

q3

Meaning : count(a) is ODD and count(b) is ODD.

Accept? : No

Accepted: "", 'aa', 'bb', 'aabb', 'abba'

Rejected: 'a', 'b', 'ab', 'aab', 'abb'

5.5 Starts with 'a', ends with 'b' (DFA)

```

M = (Q, Sigma, delta, q0, F)
Q   = { q0, q1, q2, q3 }
Sigma = { a, b }
q0   = q0
F    = { q2 }

```

```

delta(q0, a) = q1
delta(q0, b) = q3
delta(q1, a) = q1
delta(q1, b) = q2
delta(q2, a) = q1
delta(q2, b) = q2
delta(q3, a) = q3
delta(q3, b) = q3

```

DFA for strings over {a,b} that START with 'a' AND END with 'b'.

q3 is a dead/trap state ? entered if the string starts with 'b'.

q0 [START]

INITIAL state.

Meaning : No symbol read yet.

Accept? : No (empty string doesn't start with 'a')

q1

Meaning : String started with 'a'; last symbol read was 'a'.

Accept? : No (must end with 'b')

q2 [ACCEPT]

ACCEPT state.
 Meaning : String started with 'a' and currently ends with 'b'.
 Accept? : YES ?

q3

DEAD / TRAP state.
 Meaning : First symbol was 'b' ? can never satisfy 'starts with a'.
 Accept? : No (all input absorbed here)

Accepted: 'ab', 'aab', 'abb', 'abab', 'aabbb'

Rejected: '', 'a', 'b', 'ba', 'bab'

5.6 No two consecutive 1s (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{ q_0, q_1, q_2 \}$
 $\Sigma = \{ 0, 1 \}$
 $q_0 = q_0$
 $F = \{ q_0, q_1 \}$

$\delta(q_0, 0) = q_0$
 $\delta(q_0, 1) = q_1$
 $\delta(q_1, 0) = q_0$
 $\delta(q_1, 1) = q_2$
 $\delta(q_2, 0) = q_2$
 $\delta(q_2, 1) = q_2$

DFA for binary strings over $\{0,1\}$ that contain NO two consecutive 1s.
 Any occurrence of '11' sends the automaton to a permanent dead state.

q0 [START] [ACCEPT]

INITIAL & ACCEPT state.
 Meaning : Last symbol was '0' (or nothing read yet) ? safe.
 Accept? : YES ?

q1 [ACCEPT]

ACCEPT state.
 Meaning : Last symbol was '1'; a second '1' would violate the rule.
 Accept? : YES ? (one '1' is fine as long as no second follows)

q2

DEAD / TRAP state.
 Meaning : Two consecutive '1's have been seen ? rule violated.
 Accept? : No (cannot recover)

Accepted: '', '0', '1', '01', '101'

Rejected: '11', '011', '110', '111', '1011'

5.7 Every 'a' followed by 'b' (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{ q_0, q_1, q_2 \}$
 $\Sigma = \{ a, b \}$
 $q_0 = q_0$
 $F = \{ q_0 \}$

$\delta(q_0, a) = q_1$
 $\delta(q_0, b) = q_0$
 $\delta(q_1, a) = q_2$
 $\delta(q_1, b) = q_0$
 $\delta(q_2, a) = q_2$
 $\delta(q_2, b) = q_2$

DFA for strings over $\{a,b\}$ where every 'a' is immediately followed by 'b'.
 Strings ending in 'a', or containing 'aa', are rejected.

q0 [START] [ACCEPT]

INITIAL & ACCEPT state.
 Meaning : All 'a's seen so far have been followed by 'b'. Safe position.

Accept? : YES ?

q1

Meaning : Just read an 'a'; MUST see 'b' next to satisfy the rule.

Accept? : No (string cannot end here)

q2

DEAD / TRAP state.

Meaning : An 'a' was NOT immediately followed by 'b' ? rule violated.

Accept? : No

Accepted: ", 'b', 'ab', 'abb', 'abab'

Rejected: 'a', 'aa', 'ba', 'aba', 'aab'

5.8 String length divisible by 3 (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{ q_0, q_1, q_2 \}$

$\Sigma = \{ 0, 1 \}$

$q_0 = q_0$

$F = \{ q_0 \}$

$\delta(q_0, 0) = q_1$

$\delta(q_0, 1) = q_1$

$\delta(q_1, 0) = q_2$

$\delta(q_1, 1) = q_2$

$\delta(q_2, 0) = q_0$

$\delta(q_2, 1) = q_0$

DFA for strings over $\{0,1\}$ whose LENGTH is divisible by 3.

States track (length so far) mod 3. Every symbol increments the count.

q0 [START] [ACCEPT]

INITIAL & ACCEPT state.

Meaning : Characters read so far = 0 (mod 3). (0, 3, 6, ... symbols)

Accept? : YES ?

q1

Meaning : Characters read so far = 1 (mod 3). (1, 4, 7, ... symbols)

Accept? : No

q2

Meaning : Characters read so far = 2 (mod 3). (2, 5, 8, ... symbols)

Accept? : No

Accepted: ", '000', '101', '111000', '010101'

Rejected: '0', '01', '1010', '10101', '1010101'

5.9 Odd number of 1s (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{ q_0, q_1 \}$

$\Sigma = \{ 0, 1 \}$

$q_0 = q_0$

$F = \{ q_1 \}$

$\delta(q_0, 0) = q_0$

$\delta(q_0, 1) = q_1$

$\delta(q_1, 0) = q_1$

$\delta(q_1, 1) = q_0$

DFA for binary strings over $\{0,1\}$ with an ODD number of 1s.

States track parity of the count of 1s. Reading '0' preserves parity;

reading '1' flips parity.

q0 [START]

INITIAL state.

Meaning : Number of 1s read so far is EVEN (including zero).

Accept? : No (we need an odd count)

q1 [ACCEPT]

ACCEPT state.
 Meaning : Number of 1s read so far is ODD.
 Accept? : YES ?

Accepted: '1', '001', '100', '111', '10101'

Rejected: "", '0', '11', '00', '1001'

5.10 Strings ending with 'ab' (NFA)

$M = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{ q_0, q_1, q_2 \}$
 $\Sigma = \{ a, b \}$
 $q_0 = q_0$
 $F = \{ q_2 \}$

$\delta(q_0, a) = \{ q_0, q_1 \}$
 $\delta(q_0, b) = \{ q_0 \}$
 $\delta(q_1, b) = \{ q_2 \}$

NFA for strings over {a,b} that END with 'ab'.

Nondeterminism lets the automaton guess when the final 'ab' begins.

q0 [START]

INITIAL state.
 Meaning : Scanning the string; no commitment to suffix yet.
 Accept? : No

q1

Meaning : Guessed that the current 'a' is the penultimate character.
 Reached : From q0 on reading 'a' (nondeterministic branch).
 Accept? : No (need 'b' next)

q2 [ACCEPT]

ACCEPT state.
 Meaning : Matched 'ab' as the suffix of the string.
 Reached : From q1 on reading 'b'.
 Accept? : YES ?

Accepted: 'ab', 'aab', 'bab', 'abab', 'bbbab'

Rejected: "", 'a', 'b', 'ba', 'abb'

5.11 Exactly two a's (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{ q_0, q_1, q_2, q_3 \}$
 $\Sigma = \{ a, b \}$
 $q_0 = q_0$
 $F = \{ q_2 \}$

$\delta(q_0, a) = q_1$
 $\delta(q_0, b) = q_0$
 $\delta(q_1, a) = q_2$
 $\delta(q_1, b) = q_1$
 $\delta(q_2, a) = q_3$
 $\delta(q_2, b) = q_2$
 $\delta(q_3, a) = q_3$
 $\delta(q_3, b) = q_3$

DFA for strings over {a,b} with EXACTLY two 'a's.

States count the number of 'a's seen. A third 'a' sends the automaton to a permanent dead state.

q0 [START]

INITIAL state.
 Meaning : Zero 'a's seen so far.
 Accept? : No

q1

Meaning : Exactly one 'a' seen so far.

Accept? : No

q2 [ACCEPT]

ACCEPT state.

Meaning : Exactly two 'a's seen so far.

Accept? : YES ?

q3

DEAD / TRAP state.

Meaning : Three or more 'a's seen ? too many.

Accept? : No (cannot recover)

Accepted: 'aa', 'aba', 'abba', 'aabb', 'bbaab'

Rejected: '', 'a', 'aaa', 'b', 'bab'

5.12 Starts with '1', ends with '0' (DFA)

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{ q_0, q_1, q_2, q_3 \}$

$\Sigma = \{ 0, 1 \}$

$q_0 = q_0$

$F = \{ q_2 \}$

$\delta(q_0, 0) = q_3$

$\delta(q_0, 1) = q_1$

$\delta(q_1, 0) = q_2$

$\delta(q_1, 1) = q_1$

$\delta(q_2, 0) = q_2$

$\delta(q_2, 1) = q_1$

$\delta(q_3, 0) = q_3$

$\delta(q_3, 1) = q_3$

DFA for binary strings over $\{0,1\}$ starting with '1' and ending with '0'.

q3 is a dead/trap state ? entered if the string starts with '0'.

q0 [START]

INITIAL state.

Meaning : No symbol read yet.

Accept? : No

q1

Meaning : String started with '1'; last symbol read was '1'.

Accept? : No (must end with '0')

q2 [ACCEPT]

ACCEPT state.

Meaning : String started with '1' and currently ends with '0'.

Accept? : YES ?

q3

DEAD / TRAP state.

Meaning : First symbol was '0' ? can never satisfy 'starts with 1'.

Accept? : No

Accepted: '10', '100', '110', '1010', '11100'

Rejected: '', '0', '1', '01', '011'

6. NFA to DFA Conversion (Subset Construction)

The tool implements the standard subset (powerset) construction algorithm to convert any NFA into an equivalent DFA. Each DFA state represents a set of NFA states that the automaton could be in simultaneously.

Algorithm:

1. Start with the set containing only the NFA start state.
2. For each unprocessed DFA state (set of NFA states), compute transitions.
3. For each alphabet symbol, find all NFA states reachable from the current set.
4. The resulting set becomes a new DFA state (if not already seen).
5. A DFA state is accepting if it contains any NFA accept state.
6. Repeat until no new states are discovered.

Conversion: Strings containing 'aba'

Starting subset construction for NFA -> DFA conversion.

NFA start state: q_0

NFA alphabet: { a, b }

NFA accept states: { q_3 }

Step 1: Process DFA state $D_0 = \{q_0\}$

$d(D_0, a) = D_1 \leftarrow \{q_0, q_1\}$

$d(D_0, b) = D_0 \leftarrow \{q_0\}$

Step 2: Process DFA state $D_1 = \{q_0, q_1\}$

$d(D_1, a) = D_1 \leftarrow \{q_0, q_1\}$

$d(D_1, b) = D_2 \leftarrow \{q_0, q_2\}$

Step 3: Process DFA state $D_2 = \{q_0, q_2\}$

$d(D_2, a) = D_3 \leftarrow \{q_0, q_1, q_3\}$

$d(D_2, b) = D_0 \leftarrow \{q_0\}$

Step 4: Process DFA state $D_3 = \{q_0, q_1, q_3\}$

-> D_3 is an ACCEPT state (contains q_3)

$d(D_3, a) = D_3 \leftarrow \{q_0, q_1, q_3\}$

$d(D_3, b) = D_4 \leftarrow \{q_0, q_2, q_3\}$

Step 5: Process DFA state $D_4 = \{q_0, q_2, q_3\}$

-> D_4 is an ACCEPT state (contains q_3)

$d(D_4, a) = D_3 \leftarrow \{q_0, q_1, q_3\}$

$d(D_4, b) = D_5 \leftarrow \{q_0, q_3\}$

Step 6: Process DFA state $D_5 = \{q_0, q_3\}$

-> D_5 is an ACCEPT state (contains q_3)

$d(D_5, a) = D_3 \leftarrow \{q_0, q_1, q_3\}$

$d(D_5, b) = D_5 \leftarrow \{q_0, q_3\}$

=== Conversion Complete ===

DFA states: { $D_0, D_1, D_2, D_3, D_4, D_5$ }

DFA start: D_0

DFA accept: { D_3, D_4, D_5 }

Total DFA states: 6

State Mapping:

```
D0 = { q0 } [START]
D1 = { q0, q1 }
D2 = { q0, q2 }
D3 = { q0, q1, q3 } [ACCEPT]
D4 = { q0, q2, q3 } [ACCEPT]
D5 = { q0, q3 } [ACCEPT]
```

Conversion: Strings ending with 'ab'

Starting subset construction for NFA -> DFA conversion.

NFA start state: q0

NFA alphabet: { a, b }

NFA accept states: { q2 }

Step 1: Process DFA state D0 = {q0}

```
d(D0, a) = D1  <- {q0, q1}
d(D0, b) = D0  <- {q0}
```

Step 2: Process DFA state D1 = {q0, q1}

```
d(D1, a) = D1  <- {q0, q1}
d(D1, b) = D2  <- {q0, q2}
```

Step 3: Process DFA state D2 = {q0, q2}

```
-> D2 is an ACCEPT state (contains q2)
d(D2, a) = D1  <- {q0, q1}
d(D2, b) = D0  <- {q0}
```

=== Conversion Complete ===

DFA states: { D0, D1, D2 }

DFA start: D0

DFA accept: { D2 }

Total DFA states: 3

State Mapping:

```
D0 = { q0 } [START]
D1 = { q0, q1 }
D2 = { q0, q2 } [ACCEPT]
```

7. Test Results

All 12 problems include predefined test cases. The following shows the validation results for each problem.

Strings ending with '01'.....	10/10	ALL PASSED
Strings containing 'aba'.....	10/10	ALL PASSED
Binary numbers divisible by 3.....	10/10	ALL PASSED
Even number of a's and b's.....	10/10	ALL PASSED
Starts with 'a', ends with 'b'.....	10/10	ALL PASSED
No two consecutive 1s.....	10/10	ALL PASSED
Every 'a' followed by 'b'.....	10/10	ALL PASSED
String length divisible by 3.....	11/11	ALL PASSED
Odd number of 1s.....	10/10	ALL PASSED
Strings ending with 'ab'.....	10/10	ALL PASSED
Exactly two a's.....	10/10	ALL PASSED
Starts with '1', ends with '0'.....	10/10	ALL PASSED

TOTAL: 121/121 test cases passed

8. Features Summary

12 Automata Problems

10 DFA + 2 NFA covering diverse TOC concepts

Step-by-Step Simulation

Verbose transition traces for both DFA and NFA

Formal Definitions

Complete 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$

Transition Tables

Formatted tables with start/accept markers

SVG State Diagrams

Visual diagrams with circles, arrows, self-loops

NFA to DFA Conversion

Subset construction with step-by-step explanation

JFLAP Export

Download .jff files to open in JFLAP directly

State Explanations

Viva/oral exam helper for each state

Test Case Runner

121 predefined test cases with PASS/FAIL validation

Interactive Web UI

Streamlit app with dark theme, 7 tabs

CLI Mode

Full terminal-based menu system

9. Conclusion

This Automata Simulator Tool successfully demonstrates the core concepts of the Theory of Computation course through a practical implementation. The tool covers 12 language recognition problems using both DFA and NFA, provides comprehensive simulation capabilities, and includes an NFA-to-DFA converter using the subset construction algorithm.

The project serves as both a learning tool and an academic submission, with features designed to help with viva preparation, JFLAP integration, and visual understanding of finite automata.

All 121 test cases pass successfully, validating the correctness of every automaton implementation.